



南開大學
Nankai University

计算机学院

并行程序设计实验报告

多线程 NTT 实验

姓名：马禹翔

学号：2410933

专业：计算机科学与技术

2026 年 5 月 27 日

目录

1	实验目的	2
2	实验原理	2
2.1	NTT 卷积	2
2.2	蝶形运算与并行性	2
3	并行算法设计	3
3.1	pthread 采用的优化方式	3
3.2	pthread 实现	4
3.3	OpenMP 实现	5
4	实验环境	5
5	实验结果	6
6	性能分析	9
6.1	小规模性能下降	9
6.2	大规模加速效果	9
6.3	pthread 与 OpenMP 对比	10
7	实验中遇到的问题	10
7.1	旋转因子递推与并行冲突	10
7.2	同步粒度选择	10
8	结论	10

1 实验目的

本次实验选择 NTT 作为 pthread 和 OpenMP 多线程编程的研究对象。实验沿用了 Lab2 中的串行 NTT 基线、模数选择和测试规模，主要完成 pthread 和 OpenMP 两种多线程实现，并比较不同问题规模、不同线程数下的性能变化。

实验目标如下：

1. 理解 NTT 蝶形运算中的数据依赖关系，找出哪些计算单元可以放心地交给不同线程；
2. 实现串行 NTT、pthread NTT 和 OpenMP NTT，并保证三者的卷积结果完全一致；
3. 自己设计 pthread 线程同步方案，避免每一层变换中出现数据竞争；
4. 测试不同多项式规模和线程数下的运行时间、加速比和正确性；
5. 结合实验数据分析线程创建、同步、负载划分和 OpenMP 调度对性能的影响。

2 实验原理

2.1 NTT 卷积

给定两个长度为 n 的多项式

$$A(x) = \sum_{i=0}^{n-1} a_i x^i, \quad B(x) = \sum_{i=0}^{n-1} b_i x^i$$

直接卷积复杂度为 $O(n^2)$ 。NTT 将多项式从系数表示转换到模数域下的点值表示，在点值域逐点相乘后再通过逆变换恢复系数，复杂度降为 $O(n \log n)$ 。

本实验使用的模数和原根为

$$p = 7340033 = 7 \times 2^{20} + 1, \quad g = 3$$

这个模数可以支持本实验规模下的二次幂长度 NTT，而且也是 Lab2 中采用的 NTT 友好模数，因此两次实验的输入设置和正确性基准可以保持一致。

2.2 蝶形运算与并行性

迭代式 NTT 的核心循环如下：

串行 NTT 核心循环

```
1 for (int len = 2; len <= n; len <= 1) {  
2     int half = len >> 1;
```

```

3   int wlen = qpow(g, (p - 1) / len);
4   for (int base = 0; base < n; base += len) {
5       int w = 1;
6       for (int k = 0; k < half; k++) {
7           int u = a[base + k];
8           int v = a[base + k + half] * w % p;
9           a[base + k] = (u + v) % p;
10          a[base + k + half] = (u - v + p) % p;
11          w = w * wlen % p;
12      }
13  }
14 }

```

分析 NTT 循环可以发现,同一层中每个蝶形运算只读写 $a[\text{base}+k]$ 和 $a[\text{base}+k+\text{half}]$ 两个位置。对于固定的 len , 不同蝶形运算覆盖的数组位置互不重叠, 因此同一层内的所有蝶形运算可以并行执行。但层与层之间存在依赖, 当前层没有全部完成时, 下一层不能提前开始。

基于这一点, 将同一层的 $n/2$ 个蝶形运算线性编号:

$$\text{task} = \text{block} \times \text{half} + k$$

其中

$$\text{block} = \lfloor \text{task} / \text{half} \rfloor, \quad k = \text{task} \bmod \text{half}$$

然后把 task 按静态轮转方式分配给各个线程。这样做比较直接, 也方便保证每个线程拿到的任务数量尽量接近。

3 并行算法设计

3.1 pthread 采用的优化方式

本实验 pthread 部分采用的是朴素多线程优化。具体来说, 没有改变 NTT 的二分蝶形结构, 也没有做四分 NTT 或多模数 CRT 合并, 而是直接利用同一层 butterfly 之间互不依赖的特点, 将同一层的蝶形运算划分给多个线程执行。这种方法实现复杂度相对较低, 也最适合与 OpenMP 版本做一一对应的性能比较。

实验文档中 NTT 选题的进阶要求主要包括多分多线程优化和 CRT 多线程优化。多分优化可以理解为把连续两层二分蝶形合并成一次四路归并, 从而减少层数和同步次数; 但它需要重新处理四点 butterfly 的旋转因子, 并且在长度不是 4^k 时要额外预处理第一层或最后一层。为了保证主实验代码的正确性和与 Lab2 串行基线的可比性, 本实验没有把四分 NTT 放入主测试表, 而是在进阶尝试中分析其可行性。CRT 多线程优化主要面向大

模数 NTT，而当前课程框架中模数参数仍以 `int` 读入，第四组大模数已经超过整型范围，所以这部分没有作为本次实现重点。

3.2 pthread 实现

pthread 版本采用的是静态线程的思想：每次 NTT 变换创建固定数量的工作线程，线程函数内部完成所有层的蝶形运算。每一层设置两个同步点：

1. 0 号线程生成当前层旋转因子表后，所有线程等待；
2. 所有线程完成当前层 butterfly 后，再同步进入下一层。

一开始我考虑直接使用 `pthread_barrier_t`，但 Windows 下不同 pthread 环境对它的支持不完全一致。为了让代码更稳妥，我最后用 `pthread_mutex_t` 和 `pthread_cond_t` 自己实现了一个可复用 barrier。

pthread 工作线程核心

```
1 for (int len = 2; len <= n; len <= 1) {
2     int half = len >> 1;
3     if (tid == 0) {
4         int wlen = mod_pow(G, (MOD - 1) / len);
5         if (invert) wlen = mod_pow(wlen, MOD - 2);
6         roots[0] = 1;
7         for (int i = 1; i < half; i++)
8             roots[i] = mul_mod(roots[i - 1], wlen);
9     }
10    barrier.wait();
11
12    for (int task = tid; task < n / 2; task += thread_count) {
13        int block = task / half;
14        int k = task - block * half;
15        int base = block * len;
16        // butterfly
17    }
18    barrier.wait();
19 }
```

这种划分方式的优点是负载比较均衡，每个线程处理的 task 数量最多只相差 1。由于每个 task 写入的位置唯一，计算主体不需要加互斥锁；只在层边界进行同步，也可以把同步开销控制在相对少的位置。

3.3 OpenMP 实现

OpenMP 实验文档中强调，可以在外层创建并行区域，避免在循环中反复创建和销毁线程。因此 OpenMP 版本没有在一层单独写 `parallel for`，而是使用一个外层 `parallel` 区域包住整个 NTT 主循环；每一层中用 `single` 生成旋转因子表，再用 `for schedule(static)` 并行化同一层的 task 循环。

OpenMP 版本和 pthread 版本采用相同的 task 编号方式，所以二者在算法层面是一致的，主要差异来自线程管理方式、同步方式和运行时调度开销。

OpenMP 核心循环

```

1 #pragma omp parallel num_threads(thread_count)
2 {
3     for (int len = 2; len <= n; len <<= 1) {
4         #pragma omp single
5         {
6             // generate
7         }
8
9         #pragma omp for schedule(static)
10        for (int task = 0; task < n / 2; task++) {
11            // butterfly
12        }
13    }
14 }
```

4 实验环境

实验平台如下：

项目	配置
操作系统	Windows
编译器	MSYS2 UCRT64 g++ 15.2.0
语言标准	C++17
编译选项	-O2 -std=c++17 -fopenmp -pthread
测试线程数	1, 2, 4, 8
测试规模	1024, 4096, 16384, 65536, 262144

表 1: 实验环境与参数

编译和运行命令如下：

```
1 g++ -O2 -std=c++17 -fopenmp src/main.cpp -o lab3_ntt_omp.exe -pthread
2 ./lab3_ntt_omp.exe 8
```

测试数据的生成方式与 Lab2 保持一致，使用固定随机种子生成多项式系数：

$$a_i, b_i \in [0, p)$$

测试规模同样选取 1024, 4096, 16384, 65536, 262144。为了减小单次运行波动，小规模重复 10 次，中等规模重复 5 次，大规模重复 3 次，并取平均值作为最终时间。

5 实验结果

表 2 是本机测得的不同规模和线程数下的平均运行时间。时间单位为微秒，加速比都以串行 NTT 为基准。

n	线程数	串行	pthread	OpenMP	pthread 加速比	OpenMP 加速比
1024	1	123.600	126.200	123.400	0.979	1.002
1024	2	123.600	1189.700	722.500	0.104	0.171
1024	4	123.600	2274.300	1458.400	0.054	0.085
1024	8	123.600	4016.000	2516.300	0.031	0.049
4096	1	623.500	611.100	616.300	1.020	1.012
4096	2	623.500	1499.800	1155.100	0.416	0.540
4096	4	623.500	2524.100	1640.600	0.247	0.380
4096	8	623.500	4372.500	3151.200	0.143	0.198
16384	1	2987.200	2923.200	2977.000	1.022	1.003
16384	2	2987.200	3245.400	2765.800	0.920	1.080
16384	4	2987.200	3429.800	2781.800	0.871	1.074
16384	8	2987.200	5340.200	3897.600	0.559	0.766
65536	1	13825.400	14011.400	13902.400	0.987	0.994
65536	2	13825.400	10154.400	10055.800	1.362	1.375
65536	4	13825.400	9059.000	8287.200	1.526	1.668
65536	8	13825.400	10643.000	8319.200	1.299	1.662
262144	1	64251.667	62643.667	62951.333	1.026	1.021
262144	2	64251.667	39206.333	40309.667	1.639	1.594
262144	4	64251.667	31729.667	30440.667	2.025	2.111
262144	8	64251.667	30792.000	26707.000	2.087	2.406

表 2: 串行、pthread 与 OpenMP NTT 性能对比

从正确性上看，全部测试中 pthread 和 OpenMP 的输出都与串行版本完全一致。

```
MOD=7340033, primitive_root=3
n,threads,serial_us,pthread_us,openmp_us,pthread_speedup,openmp_speedup,pthread_ok,openmp_ok
1024,1,384.300,343.000,309.800,1.120,1.240,yes,yes
1024,2,384.300,5923.700,4363.300,0.065,0.088,yes,yes
1024,4,384.300,8422.900,5386.100,0.046,0.071,yes,yes
1024,8,384.300,12579.200,8506.800,0.031,0.045,yes,yes
4096,1,1505.500,1502.100,1501.500,1.002,1.003,yes,yes
4096,2,1505.500,3729.300,3065.200,0.404,0.491,yes,yes
4096,4,1505.500,6763.900,4791.500,0.223,0.314,yes,yes
4096,8,1505.500,13145.300,8865.600,0.115,0.170,yes,yes
16384,1,7112.200,7169.800,7127.800,0.992,0.998,yes,yes
16384,2,7112.200,6581.200,6669.800,1.081,1.066,yes,yes
16384,4,7112.200,8523.600,6913.400,0.834,1.029,yes,yes
16384,8,7112.200,15494.600,10985.600,0.459,0.647,yes,yes
65536,1,33320.200,32861.600,33166.200,1.014,1.005,yes,yes
65536,2,33320.200,22290.800,21454.800,1.495,1.553,yes,yes
65536,4,33320.200,19922.200,17933.400,1.673,1.858,yes,yes
65536,8,33320.200,33727.000,18868.000,0.988,1.766,yes,yes
262144,1,152160.000,150314.667,150386.000,1.012,1.012,yes,yes
262144,2,152160.000,85218.000,88223.667,1.786,1.725,yes,yes
262144,4,152160.000,68857.000,64981.333,2.210,2.342,yes,yes
```

图 1: 本地平台运行结果截图（一）

```
MOD=7340033, primitive_root=3
n,threads,serial_us,pthread_us,openmp_us,pthread_speedup,openmp_speedup,pthread_ok,openmp_ok
1024,1,249.500,250.000,375.125,0.998,0.665,yes,yes
1024,2,249.500,3000.125,374.625,0.083,0.666,yes,yes
1024,4,249.500,6442.625,248.625,0.039,1.004,yes,yes
1024,8,249.500,11940.625,374.375,0.021,0.666,yes,yes
4096,1,1498.625,1499.625,1624.500,0.999,0.923,yes,yes
4096,2,1498.625,3689.250,1499.375,0.406,0.999,yes,yes
4096,4,1498.625,6375.875,1562.875,0.235,0.959,yes,yes
4096,8,1498.625,11567.875,1499.125,0.130,1.000,yes,yes
16384,1,7249.500,7128.750,6999.500,1.017,1.036,yes,yes
16384,2,7249.500,6750.250,7128.000,1.074,1.017,yes,yes
16384,4,7249.500,7499.750,7250.000,0.967,1.000,yes,yes
16384,8,7249.500,13087.250,6989.250,0.554,1.037,yes,yes
65536,1,33253.250,33378.000,33380.250,0.996,0.996,yes,yes
65536,2,33253.250,21325.500,33215.500,1.559,1.001,yes,yes
65536,4,33253.250,18412.500,33255.500,1.806,1.000,yes,yes
65536,8,33253.250,29012.000,33377.500,1.146,0.996,yes,yes
262144,1,150281.500,150274.000,150000.000,1.000,1.002,yes,yes
262144,2,150281.500,81965.000,148498.500,1.833,1.012,yes,yes
262144,4,150281.500,62001.500,148761.000,2.424,1.010,yes,yes
```

图 2: 本地平台运行结果截图（二）

```
[s2410933@master_ubss1 ntt]$ sh test.sh 2 2
Submitted job with ID: 28773.master_ubss1
;;[1] 17:34:58 [SUCCESS] node1_ubss1
[2] 17:35:00 [SUCCESS] master_ubss7

Authorized users only. All activities may be monitored and reported.

Authorized users only. All activities may be monitored and reported.
[1] 17:35:01 [SUCCESS] node1_ubss1
[2] 17:35:02 [SUCCESS] master_ubss7
poly result correct
average latency for n = 4 p = 7340033 : 0.944367 (ms)
poly result correct
average latency for n = 131072 p = 7340033 : 32.3997 (ms)

Authorized users only. All activities may be monitored and reported.
```

图 3: SSH 平台运行结果截图

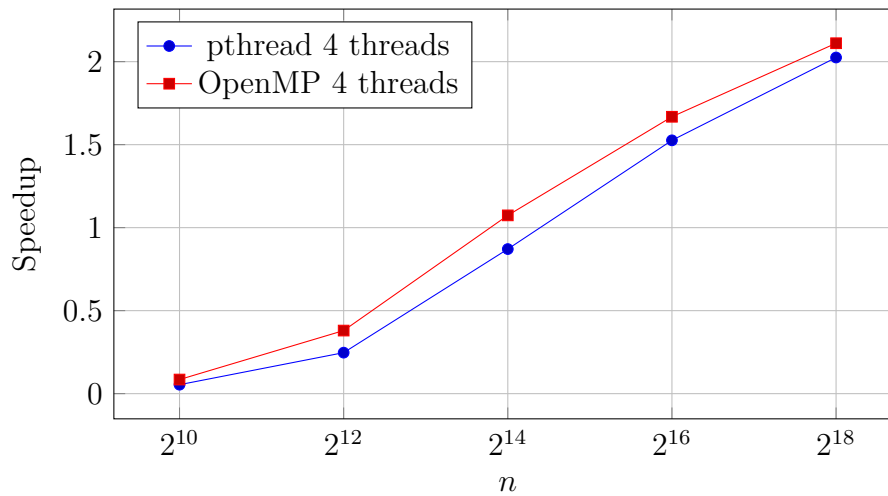
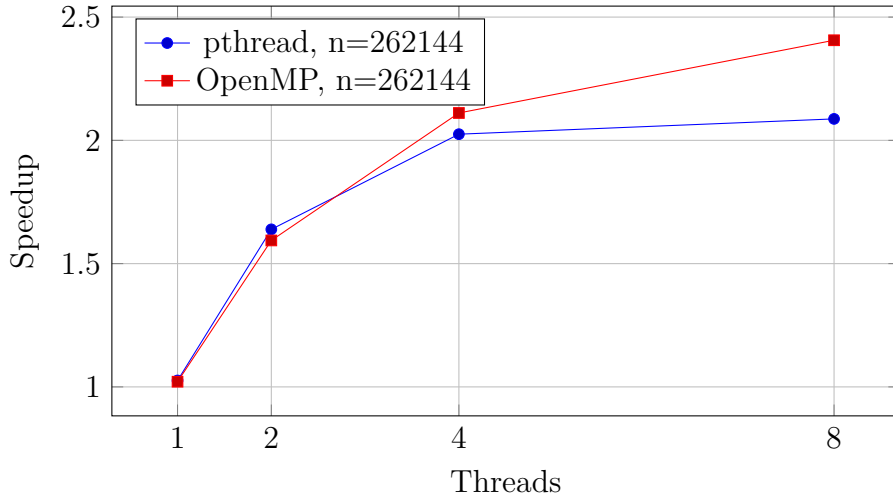


图 4: 4 线程下 pthread 与 OpenMP 加速比

图 5: $n = 262144$ 时不同线程数加速比

6 性能分析

6.1 小规模性能下降

在 $n = 1024$ 和 $n = 4096$ 时，多线程版本明显慢于串行版本。仔细想是合理的：这两个规模下 NTT 每一层的计算量并不大，而 pthread 版本在每一层需要两次 barrier 同步，OpenMP 版本也需要在每一层完成 single 与 for 的同步。线程同步、运行时调度和 cache 抖动的开销超过了并行计算带来的收益。

这也说明，并行化并不是无条件有效的。当任务粒度太小时，额外开销会主导总时间，串行执行反而更合适。

6.2 大规模加速效果

当 n 增大到 65536 和 262144 后，每一层包含的 butterfly 数量明显增加，线程同步开销被更多计算摊薄。 $n = 262144$ 时，pthread 8 线程加速比达到 2.087，OpenMP 8 线程达到 2.406，这说明同层 butterfly 并行确实能够利用多核资源。

不过 pthread 8 线程在 $n = 65536$ 和 $n = 262144$ 下没有继续显著提升。可能原因包括：

1. 每一层都需要同步，线程数增加后等待最慢线程的代价提高；
2. 多线程同时读写同一数组的不同位置，内存带宽和 cache 一致性开销增加；
3. NTT 的后几层单个 block 较大，访问跨度变化会影响 cache 局部性；
4. pthread 版本每次 NTT 变换显式创建线程，虽然避免了每层创建线程，但仍有一定线程管理成本。

6.3 pthread 与 OpenMP 对比

这次写完两版以后，我对 pthread 和 OpenMP 的差异感受更明显。pthread 的优点是控制更直接，可以明确安排 barrier 和 task 分配策略；缺点是代码量更大，线程生命周期、同步和错误处理都需要手工管理。OpenMP 的代码明显更简洁，运行时线程池和调度机制也比较成熟，因此在本实验大规模数据下表现略优。

从实验结果看，OpenMP 在 $n = 65536$ 、 $n = 262144$ 时整体优于 pthread，尤其是 8 线程下更明显。一个重要原因可能是 OpenMP 版本按照实验文档建议，把并行区域放在 NTT 主循环外部，减少了反复 fork-join 的开销；而当前 pthread 版本是在每次 NTT 变换中重新创建工作线程。

7 实验中遇到的问题

7.1 旋转因子递推与并行冲突

串行版本中每个 block 内的旋转因子通过变量 w 递推。如果直接并行化最内层循环，不同线程会破坏 w 的递推关系。为了解决这个问题，程序在每一层预先生成旋转因子表 `roots[k]`，每个 task 直接读取对应的旋转因子，从而让 butterfly 之间真正变成独立任务。

7.2 同步粒度选择

如果每个 block 或每个 butterfly 都使用锁，会产生非常高的同步开销。根据 NTT 的依赖关系，其实只需要在层与层之间同步即可。因此最终采用每层两个 barrier：一个保证旋转因子表生成完成，另一个保证本层 butterfly 全部完成。

8 结论

本次实验完成了 NTT 卷积的串行、pthread 和 OpenMP 三种实现，并在多组规模和线程数下进行了正确性与性能测试。根据实验结果，可以得到以下结论：

1. NTT 同一层的 butterfly 操作互不冲突，适合进行线程级并行；
2. 多线程版本在所有测试中都与串行结果完全一致；
3. 小规模下线程同步和调度开销会超过计算收益，导致多线程变慢；
4. 大规模下 pthread 和 OpenMP 均取得明显加速， $n = 262144$ 时 OpenMP 8 线程加速比达到 2.406；
5. OpenMP 代码更简洁，在本实验环境中大规模性能也略优；pthread 版本提供了更细粒度的控制，但还可以通过跨 NTT 调用复用线程池继续优化。

总体来看，NTT 的多线程优化重点不只是“把循环拆给多个线程”，还需要在并行粒度、同步开销和内存访问之间做权衡。只有当单层 butterfly 数量足够大时，多线程才能稳定带来性能收益。